

ANNEXURE

Architecture and evidence

The technical dossier behind the six pages above: the system in layers, the mechanism that makes removal detectable, the basis on which every test figure was measured, and the limits we state before they are asked for.

ANNEXURE A

Why removal becomes visible

A signature is a statement about a single document. A chain is a statement about a sequence. The difference only appears when something is taken out.



Each entry commits to the hash of the one before it, so an entry cannot be removed from the middle without breaking every link that follows — and the break is localised to a sequence number, not merely detected.

Deleting a record from a pile of independently signed certificates leaves the survivors verifying perfectly. Deleting one from a hash-linked ledger breaks the chain and names the break.

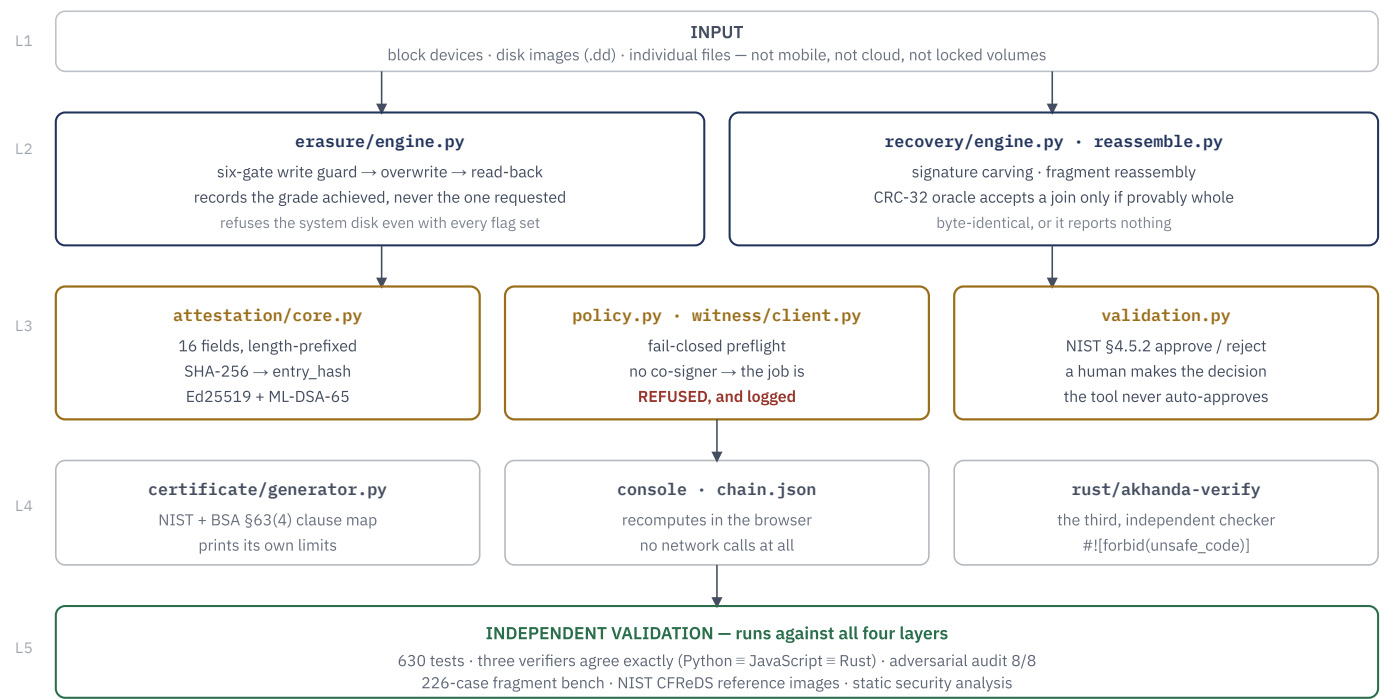
WHAT GETS SIGNED — SIXTEEN FIELDS

entry_version · chain_id · seq · prev_hash · op_type · target_ref · timestamp · method · result_hash · operator_decl · concur_decl · custody_tier · outcome · presence_ref · operator_key_id · tool_ref

Each is encoded as len(field) || field before hashing, so no two different records can produce the same bytes to sign. custody_tier and outcome sit inside the signature — which is why a wipe whose read-back failed cannot be presented as a clean one.

ANNEXURE B

Five layers, and what each one refuses to do



What each layer refuses to do is as load-bearing as what it does. Layer 2 refuses to write to a system disk and refuses to record a grade it did not achieve; layer 3 refuses to proceed without a reachable co-signer and refuses to auto-approve its own validation. **No layer calls a model at runtime** — the tool is deterministic, offline and reproducible.

DEPLOYMENT — TWO LAPTOPS, ONE HOTSPOT

LAPTOP A — THE WORKER

The tool, the operator key, the ledger and the console. It performs every operation.

LAPTOP B — THE WITNESS

One program, its own separate key, its own record of what it signed, and **no case files at all** — so anyone who breaks into it gets a key with nothing worth stealing beside it.

ANNEXURE C

What each test was measured *against*

"We ran 630 tests" is not a claim until you say what a test was measured against and what would have made it fail. This is the basis for every figure on page 4.

CLASS	REFERENCE / ORACLE	PASS CRITERION
Unit & property the 630	The written specification in <code>ENGINEERING_RULES.md</code> plus the closed vocabularies the standards define (Clear/Purge/Destroy; four custody tiers; four outcome values).	A real assertion or an expected-raise. A test with neither is not counted anywhere.
Reference data	NIST CReDS — public reference disk images with known planted files. <i>We did not author the ground truth.</i>	SHA-256 of the carved file = SHA-256 of the file NIST planted. Byte-identical, or it does not count.
Differential	Three independent implementations in languages with different string and integer semantics.	Python $\equiv$ JavaScript $\equiv$ Rust — identical entry hashes and identical verdicts.
Adversarial audit	Each claim's own negation — the audit actively tries to make each claim false.	<code>SURVIVED</code> only when falsification fails <i>for a stated, printed reason</i> .
Regression pin	Real observations from earlier runs, pinned as fixtures.	All reproduce; on drift the gate fails and a human decides which side is right .
Benchmarks	Generated images with <i>planted</i> ground truth.	Precision and recall computed, never estimated or extrapolated.

THE ORACLE PROBLEM — AND WHY VERIFICATION IS WRITTEN THREE TIMES

The hardest question in testing a hashing specification is *how do you know the expected value is right?* If a test computes the expected hash using the code under test, it proves only self-consistency — which a subtly wrong implementation also has. So expected values come from three independent implementations, and a test asserts they agree. Two implementations by one author can share a misreading and agree while both are wrong; a third, in a different language, catches that.

WHY "0 SKIPPED" IS A LOAD-BEARING NUMBER

A skipped test looks green in a summary and proves nothing. Here a skip is a non-pass everywhere: the runner reports SKIPPED as its own category and never folds it into PASS. **Skipped is not passed. Not-run is not passed either.** A separate mechanical check walks every collected test function and confirms it carries a real assertion — a suite can be large and still hollow.

MEASURED RESULTS, WITH THEIR EVIDENCE FILE

CHECK	RESULT	STATUS
Full suite	630 passed · 0 skipped · 0 failed	MEASURED
Automated gate — <code>run_all.py</code>	11 / 11 GREEN	RUN_ALL.JSON
Adversarial audit	8 / 8 survived	ADVERSARIAL_AUDIT.JSON
Three verifiers agree	identical hashes and verdicts	MEASURED
Tamper localisation, 500 entries	28 ms · names the exact seq	CHAIN_BENCHMARK.JSON
Fragment bench, 226 cases	140/140 byte-identical · 58/58 correctly refused	FRAGMENT_BENCH.JSON
CReDS L0 / L1 (contiguous)	100% precision · 100% recall	MEASURED
GB-scale carve, planted truth	1 GB @ 179 MB/s · 100% / 100%	STRESS_SCALE.JSON
Static security analysis	0 HIGH severity	BANDIT.JSON
Dual-party signing on hardware	confirmed, 3 Sep 2026	G1_CROSS_MACHINE.JSON
CReDS L4 (mixed fragmentation)	40.0% precision · 66.7% recall	BOUNDED
CReDS L2 / L3 (no ground truth)	0.0% precision, plain carver	SEE NOTE
Purge on SSD · flash wipe · bad sectors	written, fault-injection tested	NOT TESTED

Note on CReDS L2 / L3. Those two images carry no ground-truth manifest in our harness, so precision there is computed against *zero known files* — anything reported counts against it by construction. The controlled measurement of fragmented recovery is the 226-case bench, which *is* ground-truthed and reports 140/140 byte-identical with zero false positives. We publish the raw line rather than omitting it,

ANNEXURE D

Honest bounds, and how we score our own confidence

A gap that is written down is a limitation. A gap that is not written down is an overclaim waiting for somebody else to find. Each of these is stated before it is asked for, and several are printed on the certificate itself.

- CUSTODY TIER**

The tier reached is `WITNESS_COSIGNED` , never `FULL_CUSTODY` . The presence device is designed and not built, and the tool refuses to print the higher label.
- DESTRUCTIVE SANITISATION**

NIST `Purge` on an SSD, a real flash wipe and genuine bad-sector handling are **NOT TESTED**. The code is written and tested against injected faults; we could not obtain media we were permitted to destroy, so we marked it not tested rather than claim it. A fault-injection test cannot establish a physical fact.
- WHAT THE LEDGER PROVES**

It proves nothing was *altered* after being written. It cannot prove nothing was *withheld* before it was ever written. No register in the world can. Recording refused attempts narrows the gap; nothing closes it.
- MEDIA PHYSICS**

USB flash and SD cards cannot reach NIST `Purge` — no ATA Secure Erase, no NVMe Sanitize, and a translation layer that keeps blocks no host can address. On that media the tool achieves `Clear` , and says `Clear` .
- EXTERNAL ANCHORING**

OpenTimestamps provides the same class of cryptographic guarantee that courts increasingly accept. It is **not itself legally recognised in Indian courts**, and we do not say it is.
- TIMESTAMPS, KEYS, ACCESS**

Times come from the local clock. Signing keys are files on disk with optional passphrase encryption at rest. There is no access control on who may run the tool. All three are deferred and disclosed, not solved.
- SCOPE**

Block storage only. We probed a real mobile device read-only and measured that it presents as MTP with no block device to address — mobile is a different evidence class, and we say so rather than implying coverage.

HOW CONFIDENCE IS SCORED, NOT ASSERTED

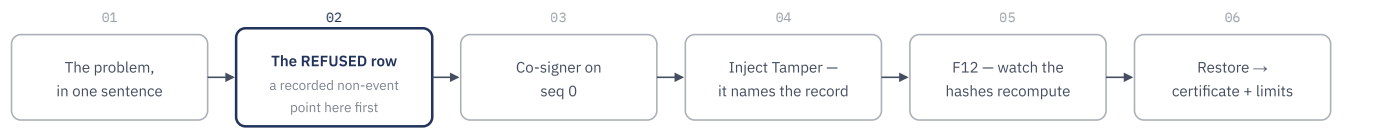
"We are confident" is not something an evaluator can check. Confidence in this project is computed in odds form from explicit likelihood ratios, each labelled *measured* (from a generated evidence file or a dated primary source) or *reasoned* (an argument, weighted lower). Evidence gathered by the same method is combined by **geometric mean rather than multiplied**, because ten tests written by one author in one afternoon are close to one piece of evidence, not ten — skipping that step is how people talk themselves into 99%. Known-unchecked falsifiers enter as ratios below 1 and are allowed to drag a posterior below its prior. No claim is scored above 0.995, because any claim resting on "we searched and found nothing" has an unknown denominator.

CLAIM	POSTERIOR	WHAT THE EVIDENCE WAS
The second signature is required by the standards, not invented by us	0.995	Two independent primary-source reads: NIST Rev. 2's Concurrence block and the BSA Schedule's Parts A and B.
Fragment reassembly is a genuine capability advance	0.991	Byte-identical recovery on ground-truthed data plus the control that the plain carver recovers nothing — same run, so geometric mean.
Post-quantum custody is defensible here	0.988	FIPS 204 final, a dated deprecation schedule, benchmarked in an existing dependency — deliberately penalised because the idea is already in the literature. First implementer, not first thinker.
"No existing tool chains its own operations"	0.47	Retired. Penalties for unchecked falsifiers pushed the posterior below its prior — which is the method working — so the claim was withdrawn rather than softened.

ANNEXURE E

What we show, in order

One command on the witness laptop, one on the operator laptop, then six things on screen. Every item is real output — a real carve, a real secure file erase, a real blocked attempt, a real certificate, a real tamper detection. **No internet at any point.**



The **REFUSED row** is the single most important thing on screen: an operation that did not happen, recorded permanently.

WHAT THE ONE COMMAND ACTUALLY DID	WHY IT IS IN THE DEMONSTRATION
Carved four real files out of a disk image	proves recovery returns files byte for byte
Securely erased three files and verified the overwrite	the erasure half of the problem statement, on a real filesystem
Attempted one job with the witness switched off	it was blocked , and the refusal was recorded
Issued the certificate	every field carries the clause it answers
Altered one entry and re-verified	verification failed and named the exact record

WHY THE BROWSER DOES THE CHECKING

If our own program says "this is valid," you are trusting us. When the page recomputes every fingerprint in front of you, on your own machine, using the browser's own cryptography, you are trusting arithmetic. Press F12 and watch it happen — the page makes no network calls at all.

REPRODUCE EVERY FIGURE IN THIS DOCUMENT

COMMAND	WHAT IT PRODUCES
<code>python scripts/run_all.py</code>	the full gate — 11 checks, nothing skipped silently
<code>python -m pytest tests/ -q</code>	the suite alone — 630 passed, 0 skipped
<code>python scripts/adversarial_audit.py</code>	eight attempts to prove our own claims hollow
<code>cargo test --manifest-path rust/akhanda-verify/Cargo.toml</code>	the independent Rust verifier's own suite

Akhanda · अखण्ड, "unbroken" · Syntax Squad · GITAM (Deemed to be University), Hyderabad · Smart India Hackathon 2026 · problem statement SIH26149 · sponsoring organisation the National Technical Research Organisation. Built on AetherProof, the team lead's own Apache-2.0 project published before this problem statement — disclosed provenance, not discovered.